Learning Without Curriculum: Autonomous Project-Based Skill Acquisition and the Scaffolding Dynamics of Generative AI

Ruthwik Reddy
Independent Student Researcher

2026

Abstract

Most formal education is built around a structured curriculum: topics are arranged in a step-by-step order, paced by an instructor, and tested through periodic exams. Yet outside school, many people learn complex technical skills in a completely different way—by diving straight into real projects and figuring things out as problems arise. With the rise of generative AI, this kind of independent learning has become far more accessible. Tools like large language models and code assistants can act as on-demand collaborators, helping learners break through roadblocks that once would have stopped them in their tracks. But this dynamic also creates a serious learning puzzle: when an AI can generate working code and explain concepts in seconds, it is tempting to offload the hard thinking, creating an "illusion of competence" where learners confuse the AI's ability with their own. In this paper, we look closely at how autonomous, project-based learning works in an AI-rich world. We examine the theoretical roots of non-curricular learning in the work of Piaget, Papert, and Bruner, and categorize the four main roles AI plays for students: Tool, Mediator, Teammate, and Subject Matter Expert. Drawing on Cognitive Load Theory and recent empirical studies, we look at quantitative evidence showing how structured reflection protocols significantly improve self-regulation ($F = 31.91, p < 0.001$). We also explore the ACCEL competency framework, outline how assessment must shift from final code artifacts to "process-trace" evaluation, and compare project-based learning with traditional curricula on learning outcomes, scalability, and equity. Finally, we share practical takeaways for students, teachers, tool designers, and schools.

1 Introduction

For generations, schools and universities have taught technical skills through a set curriculum (Blumenfeld et al., 1991). A syllabus breaks a broad field down into smaller topics, arranges them from beginner to advanced, and tests students with quizzes, problem sets, and exams. This model is well-organized and easy to run at scale, but it often has a familiar flaw: students can pass tests on theoretical concepts while still feeling completely lost when asked to build something from scratch in the real world (Barron et al., 1998; Hmelo-Silver, 2004).

At the same time, many student builders and self-directed developers learn outside this traditional structure (Knowles, 1975; Song et al., 2026). Instead of waiting for a course to tell them what to study next, they start with a project they care about—such as building a web application, automating a workflow, or analyzing a dataset. When they hit a technical wall, they search for

documentation, ask questions online, and pick up the exact concepts needed to keep going. Rather than learning theory first and applying it later, they encounter theory right when it matters most (Resnick, 2017).

Generative AI (GenAI) has supercharged this way of learning (Kumar et al., 2024; Zheng et al., 2024). In the past, solo learners often got stuck on obscure compiler errors, outdated tutorials, or complex configurations. Without a mentor nearby, frustration could easily lead them to quit. Today, modern language models and coding assistants act as an interactive safety net. They answer questions instantly, explain strange error logs, generate boilerplate code, and walk through tough concepts step by step (Chen et al., 2026).

Yet this convenience introduces a new challenge (Sweller, 1988). When a tool can write functional code or summarize complex ideas in a few seconds, it is easy to skip the mental effort that real learning requires. If learners simply accept AI answers without understanding why they work, they risk falling into an "illusion of competence"—feeling confident while an AI is by their side, but struggling to solve problems independently when it is gone.

In this paper, we explore how autonomous, project-driven learning functions when paired with generative AI. Specifically, this study addresses two guiding research questions:

1. **How do self-directed learners pick up, organize, and test complex technical skills without a traditional curriculum?**
2. **When does generative AI genuinely help people learn, and when does it become a crutch that replaces real understanding?**

By connecting classic learning theory, classroom evidence, and practical developer frameworks, this paper explores what it actually takes to learn deeply on your own in an age of intelligent tools.

2 Theoretical Foundations: From Constructionism to Conversational AI

The idea that people can learn deeply without a pre-set curriculum is not new. It has strong roots in the constructivist psychology of Jean Piaget and the constructionist educational philosophy of Seymour Papert (Piaget, 1970; Papert, 1980).

Papert often described Piaget as the original theorist of "learning without curriculum." He pointed out that young children master remarkably complex abilities—like learning their first language, figuring out how to navigate three-dimensional space, and understanding social dynamics—long before they ever sit down in a formal classroom (Papert, 1980). Children learn these skills naturally through spontaneous interaction, curiosity, and trial-and-error with the world around them. Papert argued that traditional classrooms were an artificial compromise, invented because earlier societies lacked dynamic tools that could support individualized exploration for everyone.

To bring that natural learning into modern education, Papert created the idea of computational micro-worlds, best known through the LOGO programming turtle (Papert, 1980). Designed with a "low threshold and no ceiling," these environments gave students concrete "objects to think with." When writing code to draw shapes on a screen, students had to externalize their thinking. If the

turtle drew the wrong line, the bug was visible right in front of them, prompting them to reflect, debug, and refine their mental model (Resnick, 2017).

Later, Pea and Kurland (1984) expanded on this by studying how programming languages help learners organize and clarify their problem-solving plans. In parallel, Jerome Bruner's concept of scaffolding showed that when beginners take on tough challenges, an effective tutor or tool temporarily takes care of the parts they cannot yet handle alone (Wood et al., 1976). This keeps the learner focused on the edge of their current ability—what psychologists call the Zone of Proximal Development.

Two other learning frameworks help explain why project work is so effective. First, **Self-Regulated Learning (SRL)** (Zimmerman, 2002) describes how learners steer their own progress: setting clear goals, trying different strategies, tracking their work, and reflecting honestly when things fail. When learners understand that success comes from trying better strategies rather than inborn talent, they develop the resilience needed for ambitious projects. Second, **Situated Cognition** (Lave and Wenger, 1991) reminds us that knowing and doing cannot be separated. Students understand abstract concepts much faster when they encounter them while solving a real problem, especially when collaborating in online communities and open-source projects. Real projects give ideas an immediate purpose.

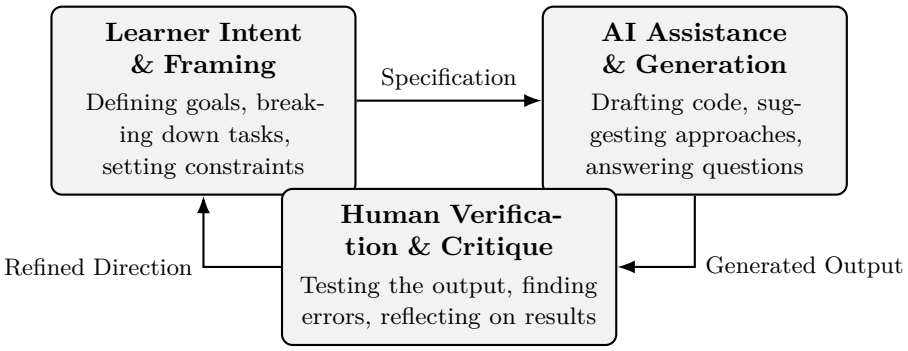


Figure 1: The Interactive Human-AI Learning Loop in Project Work.

Traditional curricula treat learning as a one-way street: teachers deliver material, and students submit static assignments. Working with modern AI feels much more like a live conversation (Zheng et al., 2024). As shown in Figure 1, the learner sets the goal and context, the AI suggests solutions or drafts code, and the learner tests, questions, and refines the output. Instead of passively following a textbook, the student becomes the active manager of their own learning environment.

3 Empirical Evidence for Project-Based and Self-Directed Learning

The theoretical case for non-curricular learning is strong; so is the empirical one. A growing body of research—meta-analyses, randomized trials, cross-national surveys, and longitudinal case studies—documents substantial and consistent learning gains from project-based and self-directed approaches.

Meta-Analytic Evidence. A comprehensive 2023 meta-analysis covering 66 classroom studies (Zheng et al., 2023) found that project-based learning clearly outperformed traditional lecture-and-test teaching, with an overall effect size of 0.44 ($p < 0.001$). The biggest advantages came in

creative thinking (0.72), computational thinking (0.63), and collaborative problem-solving. Most importantly, student interest and enthusiasm saw a major boost (0.71). When students work on projects they care about, they stay engaged longer and push through difficulty instead of tuning out.

Cultural Differences in Self-Directed Learning. Learning on your own does not feel equally natural in every educational culture. A cross-national survey across 17 countries (Guglielmino, 2011) found that readiness for self-directed learning varies widely around the world. In educational systems that have historically focused on strict teacher authority and high-stakes exams, students often feel uncomfortable when given open-ended freedom and expect instructors to tell them what to do next (Frambach et al., 2012). This is a crucial lesson for educators: self-direction is a skill that must be taught, practiced, and supported, not an automatic ability that every student already has.

Case Study: Edinburgh's SLICC Program. The University of Edinburgh's *Student-Led, Individually-Created Courses* (SLICC) show how universities can support independent project learning with clear scaffolding (Spiers et al., 2020). Students propose their own projects—such as internships, original research, or community work—set their own learning goals, and build e-portfolios with regular feedback from a tutor. Evaluations showed that participants became significantly better at explaining what they learned, connecting project work to real jobs, and treating mistakes as valuable feedback. Importantly, the program specifically reached out to students from disadvantaged and non-traditional backgrounds, proving that structured independence can expand educational opportunity.

Together, these studies show a consistent pattern: working on real projects produces deep understanding and high motivation, provided students get timely feedback and are taught how to reflect. Table 1 summarizes the key research.

4 How Learners Actually Work with AI in Self-Directed Projects

When students work on independent projects without a syllabus, they do not treat AI tools as simple search engines. Instead, they integrate them into their workflow in recognizable, structured ways (Zheng et al., 2024).

Most learners naturally split their work into different types of tasks. Repetitive, tedious, or high-friction jobs—like looking up syntax rules, writing standard boilerplate code, converting data formats, or deciphering strange terminal errors—are frequently handed off to the AI (Zheng et al., 2024; Ravi et al., 2025). Handing off these smaller hurdles lowers the barrier to getting started. A student who might have spent three frustrating hours struggling with an installation script can resolve the issue in minutes and keep their focus on the bigger picture: how their project is structured, how the pieces fit together, and how real users will interact with it.

As shown in Table 2, the way a learner views their AI tool heavily shapes what they learn from it (Zheng et al., 2024). When students treat AI purely as an answer machine, their engagement stays shallow. They quickly paste in prompts, copy the answers directly into their code, and move on. While the project might get finished, little conceptual understanding sticks.

On the other hand, when students treat the AI as a partner, mediator, or expert to be audited, learning becomes much deeper (Song et al., 2026). In that mindset, students have to:

- Express their goals clearly and define exact constraints.

Table 1: Key Empirical Studies on Project-Based and Self-Directed Learning.

Study	**Design & Sample**	**Key Findings**
Zheng et al. (2023)	Meta-analysis (66 K–12 PBL studies)	PBL yielded overall SMD ≈ 0.44 on learning outcomes. Strong gains in creative/computational thinking (SMD ≈ 0.63–0.72), problem-solving, and collaboration. Notable boost in student interest (SMD ≈ 0.713).
Guglielmino (2011)	Cross-national survey (17 countries)	Higher SDL readiness correlated with GDP per capita and individualism; lower in collectivist, high power-distance cultures.
Frambach et al. (2012)	International medical school study ($n \approx 100$–200 per site)	Cultural barriers identified: students from hierarchical systems struggled with intrinsic motivation and autonomous goal-setting.
Spiers et al. (2020)	Case study of SLICC programme (Edinburgh Univ., widening-participation cohort)	Student-designed projects improved self-assessment skills, employability thinking, and resilience; effective even for non-traditional learners.
Thomas (2000)	Literature review of PBL research	PBL and inquiry methods consistently increase deep understanding and interest; effect size varies with implementation quality and scaffolding.

Table 2: The Four Roles AI Plays for Self-Directed Learners.

Perceived Role	Typical Learner Behaviors	Impact on Learning
Tool / Assistant	Handles routine, repetitive tasks (e.g., writing boilerplate functions, debugging simple syntax mistakes, formatting data, creating mock data).	Lowers barriers to getting started and speeds up basic tasks. *Risk*: Can lead to passive copy-pasting if the code is never examined closely (Zheng et al., 2024).
Mediator	Helps bridge gaps between different technical tools or concepts; turns high-level ideas into concrete requirements; explains cryptic compiler errors in plain English.	Makes cross-disciplinary learning smoother and helps students connect new ideas to what they already know (Zheng et al., 2024).
Equal Teammate	Acts as a sounding board during brainstorming; discusses pros and cons of different design approaches; helps think through edge cases.	Encourages active discussion; prompts students to rethink their assumptions when the AI offers an alternative perspective (Song et al., 2026).
Subject Matter Expert	Answers conceptual questions on demand; explains algorithms, security practices, and system design patterns.	Provides immediate, personalized answers when stuck. *Risk*: Learners may take convincing-sounding answers at face value without checking them (Song et al., 2026).

- Read the generated code carefully to see if it actually meets their needs.
- Catch subtle errors, outdated libraries, or logical hallucinations.
- Ask follow-up questions to understand why a particular solution was chosen.

In this dynamic, technical competence is no longer just about remembering exact function names; it is about knowing how to frame problems, evaluate solutions critically, and guide complex tools toward the right outcome (Alenezi, 2026).

5 The Cognitive Offloading Dilemma: Helpful Support or Mental Crutch?

Using AI during project work creates an interesting tension that Cognitive Load Theory helps explain (Sweller, 1988). Our working memory can only handle a few new pieces of information at a time. When we offload busywork to a computer—like having a calculator do arithmetic or having an AI write repetitive code—we free up mental space to think about higher-level problem solving and system design (Sweller, 1988).

The catch is that building real understanding requires some mental effort (Song et al., 2026). Learning happens when we actively grapple with a problem, make mistakes, figure out why something failed, and fix it (Ericsson et al., 1993). When learners offload that struggle entirely to an AI, they skip the cognitive friction that helps form lasting knowledge. This creates an "illusion of competence" (Song et al., 2026): watching an AI solve a problem feels easy, so learners assume they could do it themselves—until they have to write code on a blank screen without help.

To prevent AI from becoming a mental crutch, self-directed learners need simple habits to keep themselves honest (Song et al., 2026; Zimmerman, 2002). When students are given completely free, unguided access to generative AI, the vast majority default to the path of least resistance: they copy the AI's first answer, paste it into their project, and move on if it compiles. But when they are simply asked to keep a log of what they asked, check the code for flaws, and jot down why they chose a solution, the AI turns into an effective personal coach.

This dynamic was tested directly in an empirical study of students tackling real data science projects (Song et al., 2026). As summarized in Table 3, the differences between students who used structured reflection and those who used AI freely were dramatic:

- **Students learn to catch their own errors**: The structured reflection group saw their active self-monitoring scores jump by 22% (from 3.55 to 4.32 out of 5), while the unguided group showed virtually no growth.
- **Copy-pasting drops, real understanding rises**: Guided students moved away from shallow shortcuts and showed strong gains in deep conceptual learning.
- **Scrutiny does not happen by accident**: In the unguided group, only 13% of students bothered to critique AI outputs, whereas 87% of the structured group routinely audited generated code.

Table 3: What Happens When Students Use AI With vs. Without Structured Reflection (Song et al., 2026).

Skill or Behavior	**With Structured Reflection (Prompt Logging & Review)**	**Without Structure (Free / Unguided AI Use)**	**What This Means in Plain English**
Active Self-Monitoring	Score jumped from 3.55 to 4.32 out of 5 (+22%).	Stagnant; remained flat at 3.60 out of 5.	Logging prompts trains students to monitor their own understanding and catch mistakes early.
Deep Conceptual Grasp	Statistically significant gains in understanding core concepts.	No meaningful change from starting baseline.	Students who reflect on AI answers actually learn the underlying principles.
Shallow Copy-Pasting	Sharp, substantial drop in surface-level shortcuts.	Persistent reliance on quick copy-paste tactics.	Without guidance, most students use AI just to finish the task without understanding the code.
Critiquing AI Answers	**87% of students** actively checked code for flaws.	**Only 13% of students** examined code before using it.	Learners will rarely audit AI outputs unless required to do so.
Questioning Assumptions	**80% of students** re-evaluated their own thinking.	**Only 7% of students** questioned their initial ideas.	Rethinking premises had the strongest link to lasting self-reliance ($r = 0.76$).

- **Rethinking assumptions builds real competence**: Students who practiced "double-loop reflection"—pausing to ask whether their initial question was even the right one—had the highest gains in independent problem solving ($r = 0.76, p < 0.001$).

The central lesson is clear: "learning without curriculum" does not mean learning without structure. Instead of forcing students through a pre-planned syllabus, the structure should focus on teaching them how to question, verify, and reflect on their tools (Song et al., 2026).

6 Comparing Curriculum-Based and Project-Based Learning

To see where independent project learning fits in modern education, it helps to compare it directly with traditional curriculum instruction across three key areas: learning outcomes, scalability, and equity.

Table 4: Overview of Key Studies on Project-Based Learning and AI Scaffolding.

Study	**Method**	**Learner Context**	**Key Takeaways**
Blumenfeld et al. (1991)	Classroom study with surveys	6th-grade Science	Project learning maintained high student interest and conceptual grasp, though covering individual textbook topics took longer.
Barron et al. (1998)	Multi-cohort intervention study	7th–8th-grade Science	Students in project environments outperformed peers on conceptual understanding and real-world problem-solving tasks.
Farmer et al. (2019)	Mixed methods (interviews, field observations)	Elementary club students	Tackling real environmental issues built a strong sense of personal agency and helped students apply math and reading naturally.
Zheng et al. (2023)	Meta-analysis (66 studies)	K–12 STEM	Project-based learning yielded an overall effect size of 0.44, with major gains in creative thinking (0.72) and interest (0.71).
Kumar et al. (2024)	Randomized Controlled Trial (RCT)	Undergraduate CS ($N \approx 250$)	Providing students with an LLM-guided reflection coach raised both confidence and final exam scores by +4.0 percentage points.
Ravi et al. (2025)	Co-design sessions	K–12 Teachers	Educators saw great value in AI for planning projects and tailoring feedback, but voiced valid concerns about over-reliance.

6.1 Learning Outcomes and Depth

Standard curricula excel at broad, even coverage of fundamentals. If you need to make sure every student touches each topic on a standardized list, a traditional class is well-suited for the job.

However, studies consistently show that project-based learning produces deeper conceptual understanding and longer retention (Blumenfeld et al., 1991; Barron et al., 1998). When learners spend equal amounts of time on projects, they understand core principles as well as or better

than peers in lecture courses, even if they cover fewer discrete bullet points (Barron et al., 1998). Traditional students often do better on exams right after a unit, but project learners retain their understanding far longer and are better at applying it to new, unfamiliar problems (Barron et al., 1998; Kumar et al., 2024). In software development, for example, getting a real app running requires connecting databases, APIs, interfaces, and security rules—something isolated homework exercises rarely simulate.

6.2 Scalability and Access

Traditional schooling scales through institutions: buildings, hired instructors, and standardized books. This approach provides structure, but it is expensive and geographically limited.

Independent project learning scales through modern digital networks: open-source software, online communities, and interactive tools. Generative AI makes personalized help far more scalable than before. A single teacher cannot sit down with 30 students at once to give each one custom, line-by-line feedback. But an AI tutor can walk thousands of students through a bug simultaneously, explain an idea in five different ways, and suggest next steps around the clock (Kumar et al., 2024; Ravi et al., 2025; VanLehn et al., 2005). That said, this access still depends on having reliable computers and fast internet.

6.3 Equity, Access, and the Need for Self-Direction

Project-based learning gives students the freedom to tackle challenges that matter to them and their communities (Farmer et al., 2019). But this freedom also brings real equity concerns.

Learning without a syllabus requires quiet study time, self-discipline, and reliable computer hardware. Students who have not yet had the chance to practice self-directed habits can easily feel overwhelmed by open-ended assignments (Hmelo-Silver, 2004; Zimmerman, 2002). In addition, disparities in hardware and internet access mean some students have access to top-tier AI models and powerful machines, while others do not. As cross-cultural research highlights (Guglielmino, 2011; Frambach et al., 2012), students raised in rigid, lecture-focused school systems often need explicit coaching in goal setting and self-reflection before they feel comfortable working independently.

There is also a risk that AI tools will widen this gap if left unmanaged (Ravi et al., 2025). Confident, experienced students use AI as a high-speed collaborator to build ambitious software. Less confident students, on the other hand, often use AI merely to generate answers and get past an assignment without learning the underlying principles. To make independent learning fair and accessible, schools must follow examples like the SLICC initiative (Spiers et al., 2020): make support readily available, deliberately welcome students from all backgrounds, and treat self-direction as a skill to be taught rather than an entrance requirement.

7 The ACCEL Framework: What Engineering Mastery Looks Like with AI

When students can generate code with a prompt, what skills actually define technical expertise? The ACCEL framework (Agentic Competencies through Curricula, Collaboration, and Enduring

Learning) (Alenezi, 2026) outlines five core pillars that matter most when building with AI.

Table 5: The Five Pillars of the ACCEL Framework (Alenezi, 2026).

Competency Pillar	What It Focuses On	What Success Looks Like
P1: Intent Specification & Framing	Breaking down ambiguous problems and translating them into clear, actionable requirements and prompts.	Splits complex features into manageable steps; defines inputs, outputs, and edge cases clearly; gives the AI precise context and boundaries.
P2: Iterative Verification & Critique	Checking and testing machine-generated work carefully against real-world requirements.	Reviews generated code critically; writes automated tests; spots hallucinations, security flaws, and subtle logical errors.
P3: Workflow Orchestration	Combining different tools, scripts, AI agents, and human judgment into a working pipeline.	Connects APIs, build tools, and AI helpers into a smooth development setup; uses automated tests to catch mistakes early.
P4: Reliance Calibration	Knowing when to trust AI help and when to take over manually.	Recognizes when a problem is too nuanced or critical for AI; avoids blind trust in generated output; steps in manually when needed.
P5: Ethics, Governance & Responsibility	Taking personal accountability for the safety, privacy, and impact of what is built.	Checks for security holes, biased data, and copyright issues; takes full ownership of the final deployed system.

Interestingly, empirical studies show that when beginners struggle in AI-assisted development, it is rarely because they used the wrong keywords in a prompt. Most often, they stumble on **P1 (Intent Specification)**: they jump straight to asking for code without clearly defining what they need, what the edge cases are, or what constraints must hold true (Alenezi, 2026).

7.1 A Four-Stage Learning Path

Developing these skills happens over time through four natural stages (Alenezi, 2026):

1. **Stage 1: Building Foundations**: Learners use AI mainly as a friendly tutor and explainer. Direct code generation is kept limited so students build basic mental models and understand what the code is actually doing.
2. **Stage 2: Guided Collaboration**: The AI acts like a pair programmer. The student focuses on writing clear specifications, reviewing suggested code line-by-line, and learning to catch seeded bugs in AI outputs.
3. **Stage 3: Supervised Delegation**: The learner delegates multi-step tasks (like writing tests or refactoring files) to AI agents, acting as the supervisor who sets the acceptance criteria and reviews the results.

4. **Stage 4: System Architecture**: The learner designs and orchestrates entire systems, coordinating multiple tools and workflows while maintaining complete responsibility for security, reliability, and ethics (Chen et al., 2026).

8 Rethinking Assessment: Looking at the Process, Not Just the Final Product

Because generative AI can produce an impressive-looking web app or Python script in seconds, grading solely on the final artifact no longer makes sense (Zheng et al., 2024; Lam, 2021). A working project sitting in a folder tells an evaluator very little about whether the student genuinely understands it or merely prompted a model to write it.

To solve this, self-directed and project-based education needs to embrace **Process-Trace Evaluation** (Zheng et al., 2024; Song et al., 2026). Instead of looking only at the finished project, educators and evaluators review the trail of how it was built:

1. **Prompt Logs**: A record of the conversations between the learner and the AI, showing how the student initially framed the problem and how they iterated when things went wrong.
2. **Commit and Revision Histories**: Git logs and code diffs that reveal how drafts evolved, showing where the student modified, improved, or threw away AI suggestions.
3. **Decision Notes**: Short explanations written by the student explaining why they chose a particular library, database, or architecture over alternative options.
4. **Defect Audits**: Practical exercises showing that the learner can spot edge-case bugs, security vulnerabilities, or performance bottlenecks in AI-generated code.

Keeping Process Traces Honest Of course, students could try to game prompt logs to make their process look more thorough than it was (Zheng et al., 2024). To ensure evaluation is fair and genuine, schools and programs can pair process logs with two quick human checks:

- **Short Live Walkthroughs**: Having the student explain how their code works, why they organized it the way they did, and how they fixed their biggest bug.
- **Quick Hands-On Debugging**: Asking the student to add a small feature or fix a simulated bug on the spot without AI assistance, confirming that their understanding is real.

9 Open Questions and Practical Advice

9.1 What We Still Need to Learn

While the promise of AI-supported independent learning is exciting, there are several open questions that need more long-term research:

- **Long-term retention**: Do students who learn with AI remember concepts as well two or three years later? We need longer studies tracking learners over time.
- **Assessing non-formal learning**: How can we fairly measure what someone learns from building real projects when there are no formal exams?
- **Impact on core problem-solving**: Does relying on AI code assistants make learners better architects, or does it erode their foundational problem-solving intuition over time?
- **Model biases and outdated habits**: How do the small quirks, biases, and outdated patterns in LLM training data influence the habits of new developers?

9.2 Practical Recommendations

9.2.1 For Learners

Treat AI as a thoughtful collaborator, not an automatic answer machine (Zimmerman, 2002). Before you ask an AI to write a function, pause and think through how you would solve it first. When the AI gives you code, read through it carefully, test it against edge cases, and make sure you could explain every line to a friend. Keep track of the times the AI gets things wrong—it is one of the best ways to sharpen your own critical thinking.

9.2.2 For Educators

Shift assignments away from testing rote syntax and toward problem framing, system architecture, and code review (Alenezi, 2026). Give students the freedom to choose meaningful projects, and explicitly teach them how to inspect AI outputs, spot hallucinations, and take responsibility for what they ship.

9.2.3 For AI Tool Designers

Build tools that encourage learning rather than just pure speed (Ravi et al., 2025). Instead of only providing instant code blocks, design interfaces that prompt learners to explain their reasoning, suggest helpful debugging questions, and highlight areas where the model might be less confident.

9.2.4 For Schools and Universities

Recognize that valuable learning happens outside the classroom. Develop clear pathways for portfolio reviews, digital project badges, and Prior Learning Assessments so that motivated learners can turn real-world project experience into recognized credentials (Gallagher, 2022; Lam, 2021).

10 Conclusion

Learning without curriculum in the age of generative AI is not a shortcut; when done well, it is a demanding and rewarding path to genuine skill. Rather than removing the need for intellectual

rigor, working with AI tools demands higher levels of self-discipline, clear communication, and skeptical evaluation. Seymour Papert's vision of computers as dynamic "objects to think with" has grown into conversational, intelligent partners that can support learners wherever they are.

By pairing the natural motivation of building real projects with thoughtful self-reflection and modern evaluation, we can help learners build skills that are deep, practical, and enduring. In a world where technology moves faster than any static curriculum can follow, learning how to learn on your own may be the most important skill of all.

References

Mamdouh Alenezi. Educating the agentic engineer: Curricula, collaboration, and continuous learning in the AI era. *arXiv preprint arXiv:2607.29610*, 2026. URL `https://arxiv.org/abs/2607.29610`.

Brigid J. S. Barron, Daniel L. Schwartz, Nancy J. Vye, Allison Moore, Anthony Petrosino, Linda Zech, and John D. Bransford. Doing with understanding: Lessons from research on problem- and project-based learning. *Journal of the Learning Sciences*, 7(3-4):271–311, 1998. doi: 10.1080/10508406.1998.9672056.

Phyllis C. Blumenfeld, Elliot Soloway, Ronald W. Marx, Joseph S. Krajcik, Mark Guzdial, and Annemarie Palincsar. Motivating project-based learning: Sustaining the doing, supporting the learning. *Educational Psychologist*, 26(3-4):369–398, 1991. doi: 10.1080/00461520.1991.9653139.

Eason Chen, Ce Guan, Zhonghao Zhao, Joshua Zekeri, Afeez Edeifo Shaibu, Emmanuel Osadebe Prince, Cyuan-Jhen Wu, and A. Elshafiey. When AI agents learn from each other: Insights from emergent AI agent communities on OpenClaw for human-AI partnership in education. *arXiv preprint arXiv:2603.16663*, 2026. URL `https://arxiv.org/abs/2603.16663`.

K. Anders Ericsson, Ralf T. Krampe, and Clemens Tesch-Römer. The role of deliberate practice in the acquisition of expert performance. *Psychological Review*, 100(3):363–406, 1993. doi: 10.1037/0033-295X.100.3.363.

Rachel Farmer, Richard E. West, and Erica Rosenfeld Halverson. Community super investigators: A community-based project learning experience for elementary students. *Journal of STEM Arts, Crafts, and Constructions*, 4(1):46–60, 2019.

Janneke M. Frambach, Erik W. Driessen, Li-Chong Chan, and Cees P. M. van der Vleuten. Rethinking the globalisation of problem-based learning: How culture challenges self-directed learning. *Medical Education*, 46(8):738–750, 2012. doi: 10.1111/j.1365-2923.2012.04290.x.

Sean Gallagher. *The Future of University Credentials: New Developments at the Intersection of Higher Education and Work*. Harvard Education Press, Cambridge, MA, 2022.

Lucy Madsen Guglielmino. Self-directed learning readiness and culture: A cross-national investigation across seventeen societies. *International Journal of Self-Directed Learning*, 8(2):1–18, 2011.

Cindy E. Hmelo-Silver. Problem-based learning: What and how do students learn? *Educational Psychology Review*, 16(3):235–266, 2004. doi: 10.1023/B:EDPR.0000034022.16470.f3.

Malcolm S. Knowles. *Self-Directed Learning: A Guide for Learners and Teachers*. Association Press, New York, NY, 1975.

Harsh Kumar, Ruiwei Xiao, Benjamin Lawson, Ilya Musabirov, Jiakai Shi, Xinyuan Wang, Huayin Luo, Joseph Jay Williams, Anna N. Rafferty, John Stamper, and Michael Liut. Supporting self-reflection at scale with large language models: Insights from randomized field experiments in classrooms. In *Proceedings of the Eleventh ACM Conference on Learning @ Scale (L@S '24)*, 2024. doi: 10.1145/3657604.3662042.

Ricky Lam. Portfolio assessment for the learning and assessment of disciplinary knowledge and skills. *Assessment & Evaluation in Higher Education*, 46(7):1013–1026, 2021. doi: 10.1080/02602938.2020.1837785.

Jean Lave and Etienne Wenger. *Situated Learning: Legitimate Peripheral Participation*. Cambridge University Press, Cambridge, UK, 1991.

Seymour Papert. *Mindstorms: Children, Computers, and Powerful Ideas*. Basic Books, New York, NY, 1980.

Roy D. Pea and D. Midian Kurland. On the cognitive effects of learning computer programming. *New Ideas in Psychology*, 2(2):137–168, 1984. doi: 10.1016/0732-118X(84)90018-7.

Jean Piaget. *Genetic Epistemology*. Columbia University Press, New York, NY, 1970.

Prerna Ravi, John Masla, Gisella Kakoti, Grace C. Lin, Emma Anderson, Matt Taylor, Anastasia K. Ostrowski, Cynthia Breazeal, Eric Klopfer, and Hal Abelson. Co-designing large language model tools for project-based learning with K12 educators. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems (CHI '25)*, 2025. doi: 10.1145/3706598.3713971.

Mitchel Resnick. *Lifelong Kindergarten: Cultivating Creativity Through Projects, Passion, Peers, and Play*. MIT Press, Cambridge, MA, 2017.

Ruiwen Song, Amelia Alias, and Khairul Azhar Bin Jamaludin. Heutagogy and generative ai: An empirical investigation of self-determined learning on deep learning outcomes. *Frontiers in Education*, 11:1760298, 2026. doi: 10.3389/feduc.2026.1760298.

Adam Spiers, Simon Unsworth, Simon Boyd, and Catherine Smith. Student-led, individually-created courses (SLICCs): Enabling autonomous experiential learning and metacognitive reflection in higher education. *Journal of Perspectives in Applied Academic Practice*, 8(1):44–56, 2020. doi: 10.14297/jpaap.v8i1.401.

John Sweller. Cognitive load during problem solving: Effects on learning. *Cognitive Science*, 12(2):257–285, 1988. doi: 10.1207/s15516709cog1202_4.

John W. Thomas. A review of research on project-based learning. Technical report, The Autodesk Foundation, San Rafael, CA, 2000.

Kurt VanLehn, Collin Lynch, Kay Schulze, Joel A. Shapiro, Robert Shelby, Linwood Taylor, Don Treacy, Anders Weinstein, and Mary Wintersgill. The relative effectiveness of human tutoring, intelligent tutoring systems, and other tutoring systems. *Educational Psychologist*, 40(1):3–29, 2005. doi: 10.1207/s15326985ep4001_2.

David Wood, Jerome S. Bruner, and Gail Ross. The role of tutoring in problem solving. *Journal of Child Psychology and Psychiatry*, 17(2):89–100, 1976. doi: 10.1111/j.1469-7610.1976.tb00381.x.

Chengbo Zheng, Kangyu Yuan, Bingcan Guo, Reza Hadi Mogavi, Zhenhui Peng, Shuai Ma, and Xiaojuan Ma. Charting the future of AI in project-based learning: A co-design exploration with students. In *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems (CHI '24)*, pages 1–19, 2024. doi: 10.1145/3613904.3642807.

Lan Zheng, Wei Chen, Ronghuai Huang, and Nian-Shing Chen. Effects of project-based learning on student learning outcomes in stem: A meta-analysis of 66 studies. *Educational Research Review*, 41:100561, 2023. doi: 10.1016/j.edurev.2023.100561.

Barry J. Zimmerman. Becoming a self-regulated learner: An overview. *Theory Into Practice*, 41(2): 64–70, 2002. doi: 10.1207/s15430421tip4102_2.